

LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

PERTEMUAN 13



Disusun oleh:

Christoba Joshua Hutagalung

2311102133

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

LINK GIT

https://github.com/Christobacode/Praktikum_ABP.git

TUGAS PERTEMUAN 13

Pertemuan 13

Tugas Praktik Modul 12 & 13

Implementasi Provider dan Notifikasi pada Flutter

Deskripsi Tugas

Buatlah aplikasi Flutter sederhana yang menerapkan **State Management Provider** dan **Notifikasi**. Aplikasi cukup satu halaman yang menampilkan nilai counter dan sebuah tombol untuk menambah nilai counter.

Ketentuan

1. Gunakan **Provider** untuk menyimpan dan mengelola nilai counter.
2. Tampilkan nilai counter pada layar utama aplikasi.
3. Sediakan tombol **Tambah (+)** untuk menambah nilai counter sebanyak 1 setiap kali ditekan.
4. Setiap kali nilai counter bertambah, tampilkan notifikasi yang berisi:
 - Judul: **Counter Update**
 - Pesan: **"Nilai counter saat ini: X"** (X adalah nilai counter terbaru)
5. Notifikasi dapat menggunakan:
 - **Firebase Cloud Messaging (FCM)**, atau
 - **Local Notification** (lebih sederhana).
6. Tampilan aplikasi tidak perlu dibuat kompleks, cukup fungsional dan mudah digunakan.

Jawab :

main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter_provider.dart';
import 'notification_service.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await NotificationService.init();

  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

```

    home: const CounterPage(),
  );
}
}

class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    final counter = context.watch<CounterProvider>().counter;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Tugas Modul 13 - Mobile'),
      ),
      body: Center(
        child: Text(
          'Counter: $counter',
          style: const TextStyle(fontSize: 32),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          context.read<CounterProvider>().increment();
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}

```


Penjelasan :

Pada bagian utama aplikasi yang terdapat dalam file main.dart, alur eksekusi dimulai dengan memastikan seluruh binding sistem Flutter telah siap, dilanjutkan dengan pemanggilan fungsi inisialisasi untuk layanan notifikasi lokal. Setelah persiapan sistem selesai, kerangka utama aplikasi dibungkus menggunakan widget ChangeNotifierProvider agar state management dari CounterProvider dapat diakses secara global oleh seluruh komponen UI di dalamnya. Layar utamanya sendiri dibangun menggunakan StatelessWidget biasa yang menampilkan teks berisi nilai counter. Angka pada teks ini bersifat reaktif karena dipantau menggunakan metode context.watch. Di sudut kanan bawah layar, disematkan sebuah tombol interaktif (Floating Action Button) yang bertugas memicu penambahan angka melalui fungsi bawaan provider menggunakan metode context.read, sehingga proses pembaruan data dapat berjalan efisien tanpa perlu merender ulang keseluruhan struktur halaman.

Counter_provider.dart

```
import 'package:flutter/material.dart';
import 'notification_service.dart';

class CounterProvider with ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    NotificationService.show(_counter);
    notifyListeners();
  }
}
```

Penjelasan :

File counter_provider.dart berperan sebagai otak pengelola data (State Management) utama dalam aplikasi ini dengan memanfaatkan mixin ChangeNotifier. Di dalam kelas ini, terdapat sebuah variabel privat bernama _counter yang berfungsi untuk menyimpan angka secara aman, sementara widget lain di luar kelas hanya diizinkan untuk membaca nilainya melalui fungsi getter. Kunci utama dari file ini ada pada fungsi increment(). Setiap kali fungsi ini ditekan melalui tombol di antarmuka pengguna, ia akan mengeksekusi tiga perintah secara berurutan: menambahkan nilai variabel sebesar satu angka, mengirimkan nilai terbaru tersebut ke antarmuka layanan notifikasi agar segera ditampilkan sebagai pop-up, dan terakhir menembakkan perintah notifyListeners() agar seluruh layar UI yang terhubung segera memperbarui tampilannya secara otomatis.

Notification_service.dart

```
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class NotificationService {
  static final FlutterLocalNotificationsPlugin _notifications =
    FlutterLocalNotificationsPlugin();

  static Future<void> init() async {
    const androidInit = AndroidInitializationSettings('@mipmap/ic_launcher');

    const InitializationSettings settings = InitializationSettings(
      android: androidInit,
    );

    await _notifications.initialize(settings: settings);

    await _notifications
      .resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin
      >()
      ?.requestNotificationsPermission();
  }

  static Future<void> show(int counter) async {
    const androidDetails = AndroidNotificationDetails(
      'counter_channel',
      'Counter Notification',
      channelDescription: 'Notifikasi setiap counter bertambah',
      importance: Importance.max,
      priority: Priority.high,
    );

    const details = NotificationDetails(android: androidDetails);

    await _notifications.show(
      id: 0,
      title: 'Counter Update',
      body: 'Nilai counter saat ini: $counter',
      notificationDetails: details,
    );
  }
}
```

Penjelasan :

Modul notification_service.dart dirancang khusus sebagai pengatur lalu lintas notifikasi lokal yang berjalan di latar belakang sistem. Pada tahap awal, kelas ini memiliki fungsi init() yang bertugas menyiapkan ikon dasar Android sekaligus secara proaktif meminta izin akses (permission) kepada sistem agar aplikasi diperbolehkan memunculkan notifikasi di perangkat pengguna. Untuk aksi pemunculannya, terdapat fungsi show() yang akan menerima parameter berupa angka counter terbaru dari Provider. Fungsi ini mengonfigurasi rincian saluran notifikasi (channel) dengan tingkat kepentingan dan prioritas maksimal. Hal ini diatur

sedemikian rupa agar sistem Android langsung menembakkan notifikasi tersebut dalam bentuk heads-up (muncul menutupi sebagian kecil bagian atas layar) untuk menarik perhatian pengguna sesaat setelah angka bertambah.

Tambahkan Dependencies di pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.1.5+1  
  flutter_local_notifications: ^22.0.1  
  
  # The following adds the Cupertino Icons font to your application.  
  # Use with the CupertinoIcons class for iOS style icons.  
  cupertino_icons: ^1.0.8
```

Penjelasan:

Pada konfigurasi proyek ini, aplikasi sangat bergantung pada dua pustaka pihak ketiga utama agar dapat beroperasi sesuai dengan instruksi tugas. Pustaka pertama adalah provider, yang diintegrasikan untuk memberikan solusi pengelolaan state secara modern sehingga pertukaran data angka counter dari logic ke tampilan (UI) dapat berjalan mulus. Pustaka kedua adalah flutter_local_notifications, yang disuntikkan ke dalam sistem agar aplikasi memiliki kemampuan mandiri dalam merakit dan memunculkan pesan peringatan sistem ke layar perangkat tanpa memerlukan bantuan peladen (server) eksternal maupun koneksi internet.

Tambahkan Konfigurasi Izin Sistem Android di android/app/src/main/AndroidManifest.xml

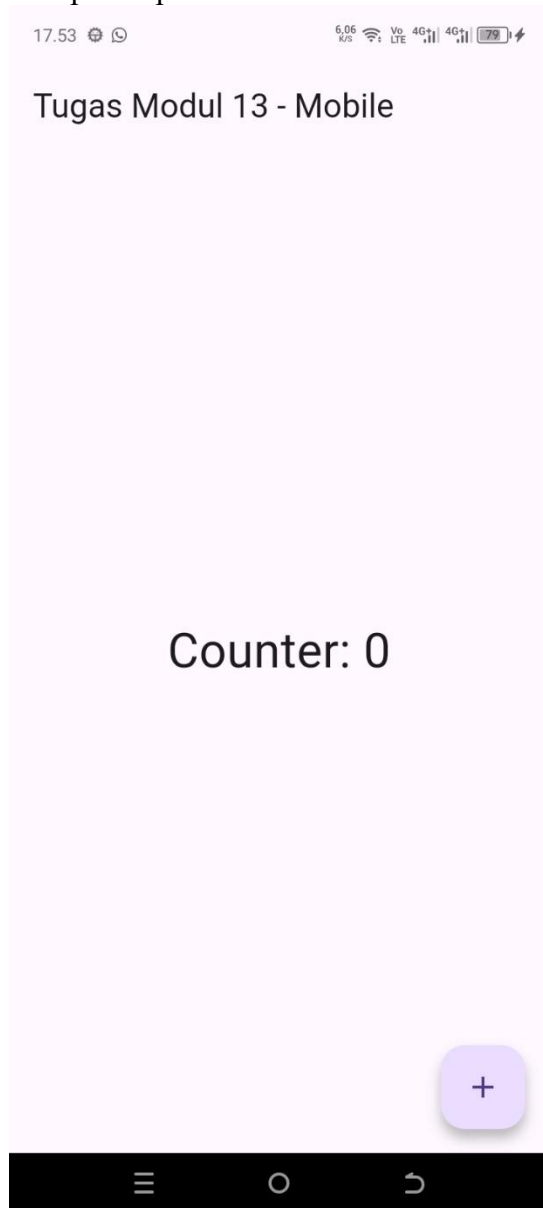
```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

Penjelasan :

Untuk melengkapi sistem keamanan Android modern, file konfigurasi manifest disesuaikan dengan menambahkan satu baris perizinan krusial, yaitu android.permission.POST_NOTIFICATIONS. Penambahan izin sistem ini merupakan protokol wajib sebagai bentuk perlindungan privasi, khususnya untuk perangkat yang menggunakan OS Android 13 ke atas. Dengan dideklarasikannya izin ini, aplikasi secara legal mendaftarkan dirinya untuk memohon hak akses mengirimkan banner kepada pengguna, sehingga sistem operasi tidak akan memblokir pop-up notifikasi yang muncul dari penekanan tombol penambah counter.

OUTPUT

Tampilan aplikasi



Saat nilai counter bertambah

